LIBRIS

Library Management System

Full Technical Documentation

Version	1.0.0
Author	Bhungani Kenil Shaileshbhai
Team	Venil Virani Nishi Macwan Maheshkumar Virani Dhruv
Date	April 2026
Platform	Python 3.10 · Flask · Jinja2
Stack	HTML5 · CSS3 · JavaScript · JSON
Tool	Trunk.io (code quality)

Table of Contents

Chapter 1

Introduction

Project overview, purpose, and intended audience

1.1 Project Overview

Libris is a full-featured, web-based Library Management System developed using Python 3.10 and the Flask micro-framework. It provides a comprehensive platform for managing a library's catalogue of books and its registered members, tracking borrowing and returning of books, searching the catalogue, and generating activity logs and statistical reports — all accessible through a clean, responsive browser-based interface.

The system requires no external database server. All data is persisted in JSON flat files in a local data/ directory, making the system easy to deploy, back up, and migrate. Libris is well-suited for small to medium-sized libraries, school libraries, office book collections, or personal book tracking.

1.2 Purpose of This Document

This document serves as the complete technical and user reference for the Libris Library Management System. It covers the system architecture, module descriptions, API route reference, data storage schemas, frontend structure, installation guide, and detailed usage instructions.

1.3 Scope & Audience

This document is intended for the following audiences:

- **Developers** — who need to understand the codebase, add features, or fix issues.
- **System Administrators** — who need to deploy, configure, and maintain the application.
- **Library Staff / End Users** — who need guidance on using the web interface.
- **Technical Reviewers** — conducting code review or project assessment.

> *Note: This document covers Libris version 1.0.0. Features or behaviours may differ in future releases.*

Chapter 2

System Architecture

High-level design, technology stack, and data flow

2.1 High-Level Architecture

Libris follows a classic Model-View-Controller (MVC) architectural pattern adapted for Flask:

- **Model Layer:** book.py, user.py, library.py — define data structures and business logic.
- **View Layer:** templates/ directory — Jinja2 HTML templates that render the user interface.
- **Controller Layer:** app.py — Flask routes that receive HTTP requests, invoke the model, and return rendered views.
- **Persistence Layer:** data/ directory — JSON files for storage (no database server required).

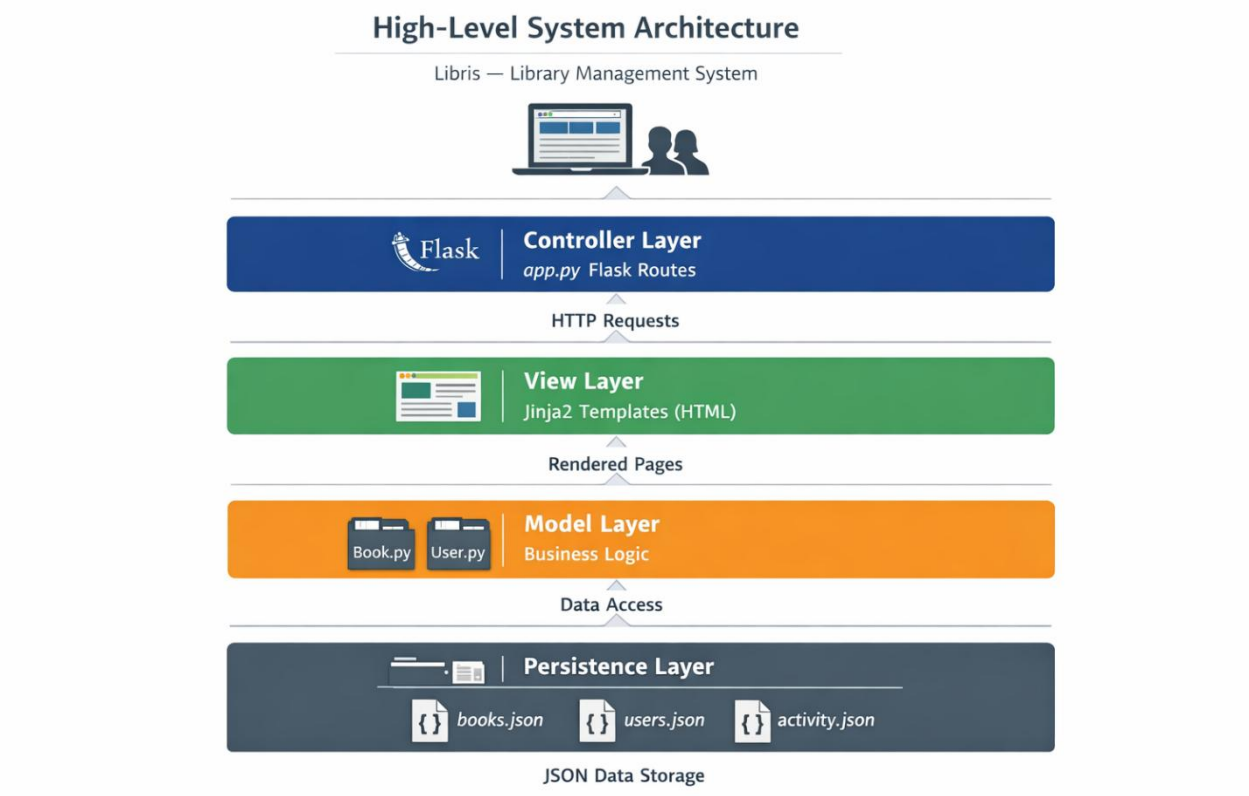

Figure 2.1 — High-Level System Architecture

2.2 Technology Stack

Layer	Technology	Purpose
Backend Language	Python 3.10	Core application logic
Web Framework	Flask	HTTP routing and request/response handling
Templating	Jinja2	Dynamic HTML rendering (built into Flask)
Frontend	HTML5 / CSS3	Page structure and global styling
Interactivity	JavaScript	Client-side validation and UI behaviour
Data Storage	JSON Files	Flat-file persistence — no DB server needed

Layer	Technology	Purpose
Code Quality	Trunk.io	Automated linting and formatting

2.3 Data Flow

The complete request-response cycle within Libris:

- **Step 1 — Browser Request:** The user navigates to a URL, generating an HTTP GET or POST request.
- **Step 2 — Route Matching:** Flask's URL dispatcher in app.py matches the URL to a view function.
- **Step 3 — Data Loading:** The view function reads the relevant JSON file into a Python list.
- **Step 4 — Business Logic:** The view function processes the data and applies changes.
- **Step 5 — Data Saving:** If data was modified, the updated list is written back to the JSON file.
- **Step 6 — Template Rendering:** Flask passes data to a Jinja2 HTML template.
- **Step 7 — HTTP Response:** The rendered HTML is returned and displayed in the browser.

Chapter 3

Project Structure

Complete file and folder reference

3.1 Root Directory

The root of the project (C:\Users\bhung\library-management\) contains:

File / Folder	Type	Size	Description
app.py	Python Script	~15 KB	Main Flask application — all routes and logic
book.py	Python Script	~2 KB	Book class definition
library.py	Python Script	~3 KB	Helper and utility functions
user.py	Python Script	~2 KB	User (Member) class definition
data/	Folder	~5 KB	JSON persistent data storage
static/	Folder	~29 KB	CSS and JavaScript assets
templates/	Folder	~40 KB	Jinja2 HTML templates
.trunk/	Folder	—	Trunk.io code quality tooling
__pycache__/	Folder	~17 KB	Python compiled bytecode cache

3.2 /data Directory

Stores all application data as plain JSON files, created automatically on first run.

File	Size	Contents
activity.json	2 KB	Array of activity log entries (borrow, return, add events)
books.json	2 KB	Array of book record objects
users.json	1 KB	Array of library member record objects

Note: activity.json is automatically capped at 100 entries. Older entries are removed as new ones are added.

3.3 /static Directory

File	Type	Size	Description
style.css	CSS Stylesheet	26 KB	Complete global styling — layout, navigation, cards, tables, forms, responsive design
script.js	JavaScript	3 KB	Client-side interactivity — form validation, dynamic dropdowns, flash message dismissal

3.4 /templates Directory

Contains 11 Jinja2 HTML templates. All extend base.html.

Template File	Size	Route(s)	Description
base.html	6 KB	All pages	Base layout — nav, header, footer, flash messages
index.html	5 KB	/	Dashboard — stats, recent books, recent activity
books.html	4 KB	/books	Full catalogue list with status indicators
add_book.html	3 KB	/books/add	Form to add a new book
book_detail.html	4 KB	/books/	Individual book info and borrow history
users.html	3 KB	/users	Member list with borrowed book counts
add_user.html	4 KB	/users/add	Form to register a new member
borrow.html	5 KB	/borrow	Borrow / return form
search.html	10 KB	/search	Search interface and results
activity.html	2 KB	/activity	Full chronological activity log
stats.html	4 KB	/stats	Genre breakdown and top borrowers

3.5 /.trunk Directory

Item	Type	Purpose
trunk.yaml	YAML Config	Main Trunk configuration — enabled linters, formatters, tools
.gitignore	Git Ignore	Trunk-specific gitignore rules
configs/	Folder	Per-tool configuration files
plugins/	Folder	Trunk plugin definitions
actions/	Folder	Custom Trunk CI actions
tools/	Folder	Trunk-managed tool binaries

Item	Type	Purpose
logs/	Folder	Execution logs from Trunk runs
out/	Folder	Output artifacts from Trunk runs
notifications/	Folder	Notification channel settings

3.6 /__pycache__Directory

Auto-generated by Python. Stores compiled .pyc bytecode. Add to .gitignore and never edit manually.

File	Size	Source
book.cpython-310.pyc	2 KB	Compiled from book.py
email_utils.cpython-310.pyc	10 KB	Compiled from email_utils.py
library.cpython-310.pyc	3 KB	Compiled from library.py
user.cpython-310.pyc	2 KB	Compiled from user.py

Chapter 4

Installation & Setup

Prerequisites, installation steps, and first launch

4.1 Prerequisites

Requirement	Version	Notes
Python	3.10+	Verified — cpython-310 bytecode in __pycache__
pip	Latest	Bundled with Python 3.10+
Flask	2.x+	The only required external package
Browser	Any modern	Chrome, Firefox, Edge, or Safari
OS	Any	Windows, macOS, or Linux

4.2 Installation Steps

Step 1 — Clone the Repository

Open a terminal or Command Prompt and run:

```
git clone https://github.com/your-username/library-management.git cd
library-management
```

Step 2 — Install Flask

Install the only required dependency:

```
pip install flask
```

Step 3 — Verify Installation

Confirm Flask installed correctly:

```
python -m flask --version
```

4.3 Running the Application

Start the Flask development server from the project root:

```
python app.py
```

Open any browser and navigate to http://127.0.0.1:5000

4.4 First Launch

On first launch, app.py automatically creates the data/ directory and initialises the three JSON data files as empty arrays. No manual setup is required.

> *Note: The Flask development server is for development only. For production, use Gunicorn with a reverse proxy such as Nginx.*

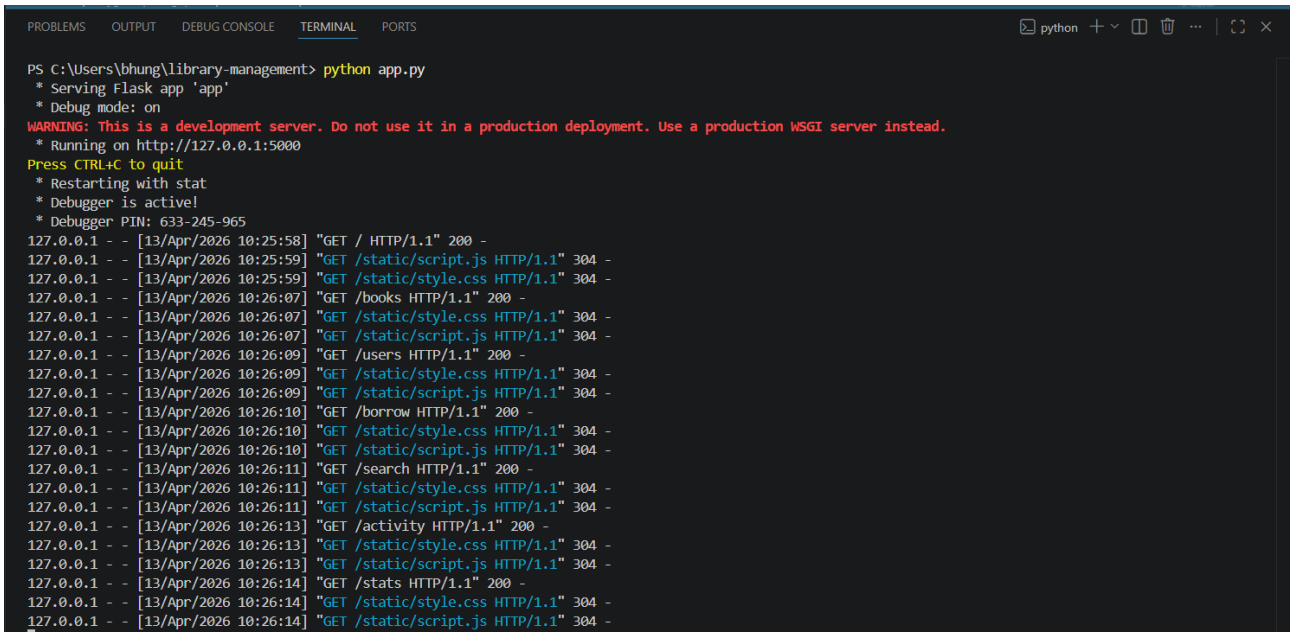

Figure 4.1 — Application startup terminal output

Chapter 5

Module Reference

Detailed reference for all Python source files

5.1 app.py — Flask Application

app.py is the main entry point and controller of Libris. It initialises the Flask application, defines all URL routes, implements data helper functions, and handles the complete HTTP request-response lifecycle.

Configuration Variables

Variable	Value	Purpose
app.secret_key	libris-secret-key-2024	Signs session cookies and flash messages
DATA_DIR	data	Directory path for JSON storage files
BOOKS_FILE	data/books.json	Path to books data file
USERS_FILE	data/users.json	Path to users data file
ACTIVITY_FILE	data/activity.json	Path to activity log file

Data Helper Functions

Function	Returns	Description
load_books()	list	Reads and returns books.json as a Python list
save_books(books)	None	Serialises and writes the books list to books.json
load_users()	list	Reads and returns users.json as a Python list
save_users(users)	None	Serialises and writes the users list to users.json
load_activity()	list	Reads and returns activity.json as a Python list
save_activity(activity)	None	Writes activity list to file (capped at 100 entries)
log_activity(type,user,book)	None	Creates and prepends a new activity log entry

5.2 book.py — Book Class

Represents a single book record. Encapsulates all attributes and provides borrow/return methods.

Constructor

```
Book(title, author, year, isbn, genre='', description='')
```

Parameter	Type	Required	Description
title	str	Yes	Full title of the book
author	str	Yes	Author name
year	str	Yes	Publication year
isbn	str	Yes	Unique ISBN — primary key
genre	str	No	Genre / category (default: empty string)
description	str	No	Short description (default: empty string)

Methods

Method	Parameters	Returns	Description
borrow(user_name)	user_name: str	None	Sets available=False, borrowed_by=user_name
return_book()	None	None	Sets available=True, borrowed_by=None
to_dict()	None	dict	Returns all attributes as dict for JSON storage
__str__()	None	str	Human-readable: 'Title by Author (Year) — Status'
__repr__()	None	str	Dev repr: 'Book(title=..., isbn=...)'

5.3 user.py — User Class

Represents a library member. Stores contact info and maintains a list of borrowed ISBNs.

Constructor

```
User(name, user_id, email='', phone='')
```

Parameter	Type	Required	Description
name	str	Yes	Full name of the member
user_id	str	Yes	Unique member identifier (e.g., U001)
email	str	No	Email address (default: empty string)
phone	str	No	Phone number (default: empty string)

Methods

Method	Parameters	Returns	Description
borrow_book(isbn)	isbn: str	None	Appends ISBN to borrowed_books if not already present
return_book(isbn)	isbn: str	None	Removes ISBN from borrowed_books if present
to_dict()	None	dict	Returns all attributes as dict for JSON storage
__str__()	None	str	'Member: Name (ID: X) — N book(s) borrowed'
__repr__()	None	str	Dev repr: 'User(name=..., user_id=...)'

5.4 library.py — Helper Functions

Standalone utility functions operating on in-memory lists. No file I/O performed.

Function	Parameters	Returns	Description
search_by_title(library, title)	list, str	list	Partial, case-insensitive title match
search_by_author(library, author)	list, str	list	Partial, case-insensitive author match
search_by_isbn(library, isbn)	list, str	dict|None	Exact ISBN lookup — single book or None
delete_book(library, isbn)	list, str	bool	Removes book in-place; returns True/False
get_available_books(library)	list	list	Returns all books where available == True
get_borrowed_books(library)	list	list	Returns all books where available == False
get_stats(library, users)	list, list	dict	Returns total, available, borrowed, utilization_pct

Chapter 6

API Route Reference

Complete HTTP endpoint documentation

6.1 Dashboard Routes

Route	Method	Template	Description
/	GET	index.html	Renders dashboard: total books, available, borrowed, total members, 5 most recent books, 6 most recent activity entries.

6.2 Book Routes

Route	Method	Template	Description
/books	GET	books.html	Displays the full book catalogue with availability status.
/books/add	GET	add_book.html	Renders the Add Book form.
/books/add	POST	Redirect	Validates form, checks duplicate ISBN, saves book, logs activity, redirects to /books.
/books/	GET	book_detail.html	Renders detail page for a single book with borrow history.
/books/delete/	POST	Redirect	Deletes a book by ISBN and redirects with flash message.

6.3 User Routes

Route	Method	Template	Description
/users	GET	users.html	Displays all registered members.
/users/add	GET	add_user.html	Renders Register Member form with existing_ids for duplicate checking.
/users/add	POST	Redirect	Validates form, checks duplicate user_id, saves member, redirects.
/users/delete/	POST	Redirect	Removes a member and redirects with flash message.

6.4 Borrow & Return Routes

Route	Method	Template	Description
/borrow	GET	borrow.html	Splits books into available and borrowed lists; renders borrow/return page.
/borrow	POST	Redirect	Reads action (borrow/return), isbn, user_id. Borrow: marks book unavailable, adds ISBN to user. Return: marks book available, removes ISBN. Logs activity. Redirects.

6.5 Search & Activity Routes

Route	Method	Template	Description
/search	GET	search.html	Case-insensitive partial match across title, author, ISBN, and genre.
/activity	GET	activity.html	Loads and renders the full activity log from activity.json.

6.6 Statistics Routes

Route	Method	Template	Description
/stats	GET	stats.html	Computes: total/available/borrowed counts, genre breakdown sorted by count, and top 5 borrowers from the activity log.

Chapter 7

User Interface & Pages

Detailed description of every page with screenshot placeholders

7.1 Dashboard (Home Page)

Route: `/`

The Dashboard is the first page a user sees. It provides an at-a-glance summary of the entire library.

Key Features:

- Four summary stat cards: Total Books, Available Books, Borrowed Books, Total Members
- A 'Recent Books' panel showing the 5 most recently added books
- A 'Recent Activity' panel showing the 6 most recent events
- Navigation bar linking to all major sections

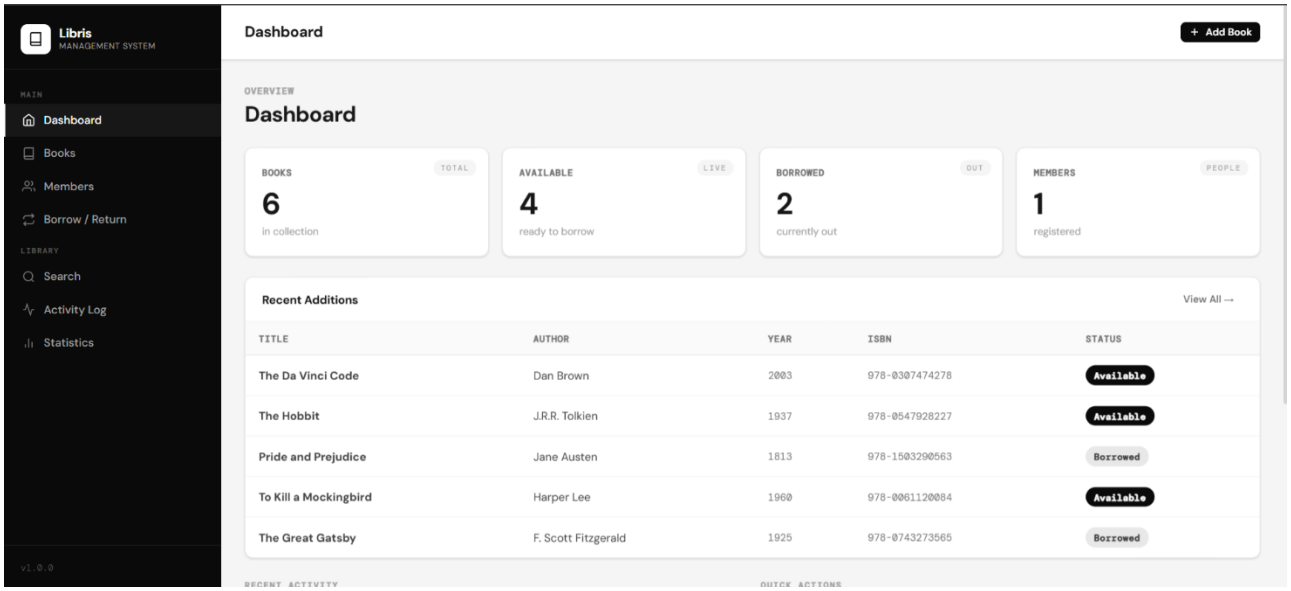

Figure 7.1 — Dashboard (Home Page)

7.2 Books Management Page

Route: `/books`

Lists every book in the catalogue with title, author, year, ISBN, genre, and availability status.

Key Features:

- Complete book catalogue in a tabular layout
- Availability status badge — Available / Borrowed
- Link to each book's detail page
- Delete button per book
- 'Add Book' navigation button

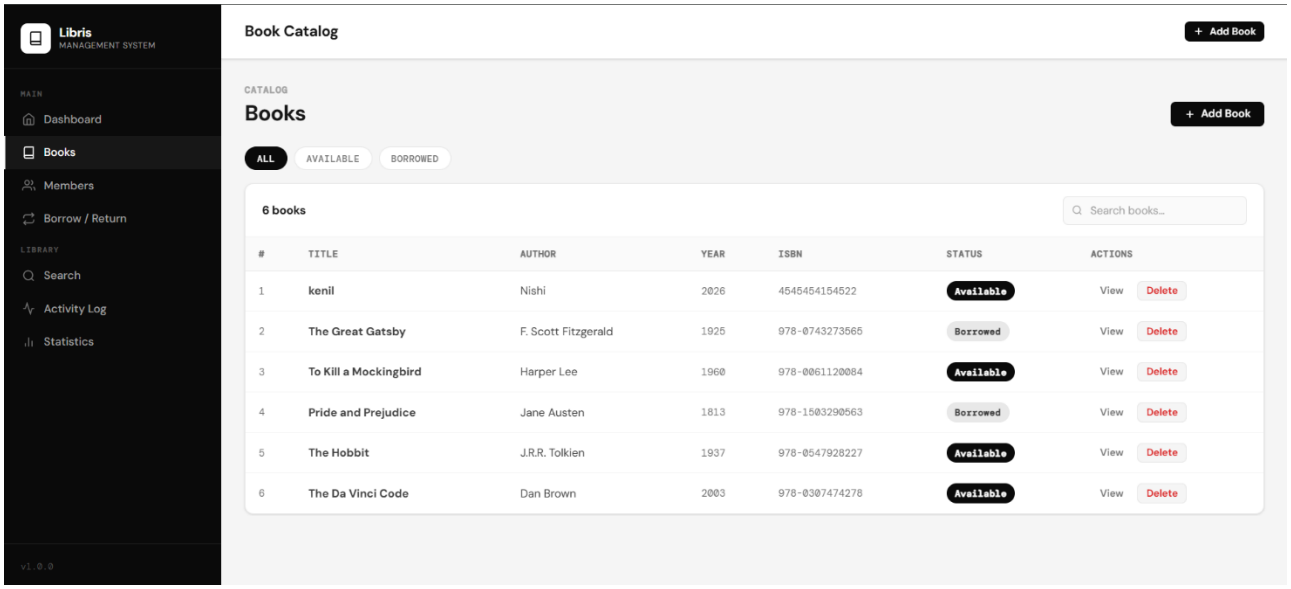

Figure 7.2 — Books Management Page

7.3 Add Book Page

Route: `/books/add`

Form for adding new books. Validates that the ISBN is unique before saving.

Key Features:

- Required: Title, Author, Year, ISBN
- Optional: Genre, Description (textarea)
- Duplicate ISBN validation — error flash if ISBN exists
- Success flash on addition

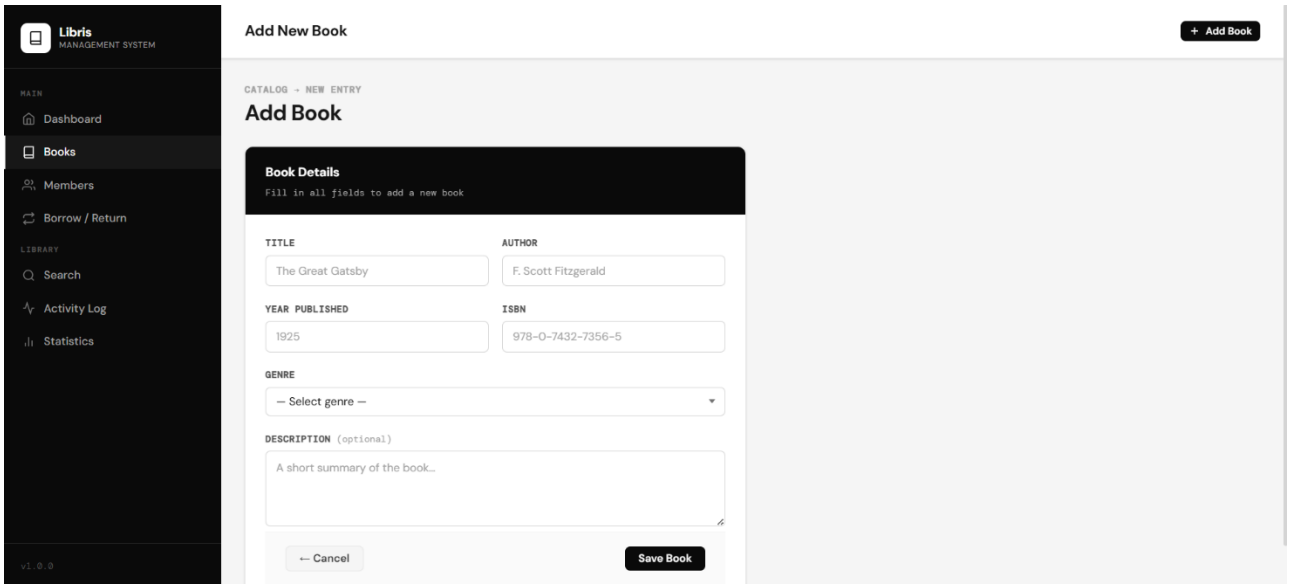

Figure 7.3 — Add Book Page

7.4 Book Detail Page

Route: `/books/`

Shows all information about a single book and its complete borrowing history.

Key Features:

- All book fields displayed
- Current availability status and borrower name
- Chronological borrow/return history (last 10 entries)
- Delete button for the book

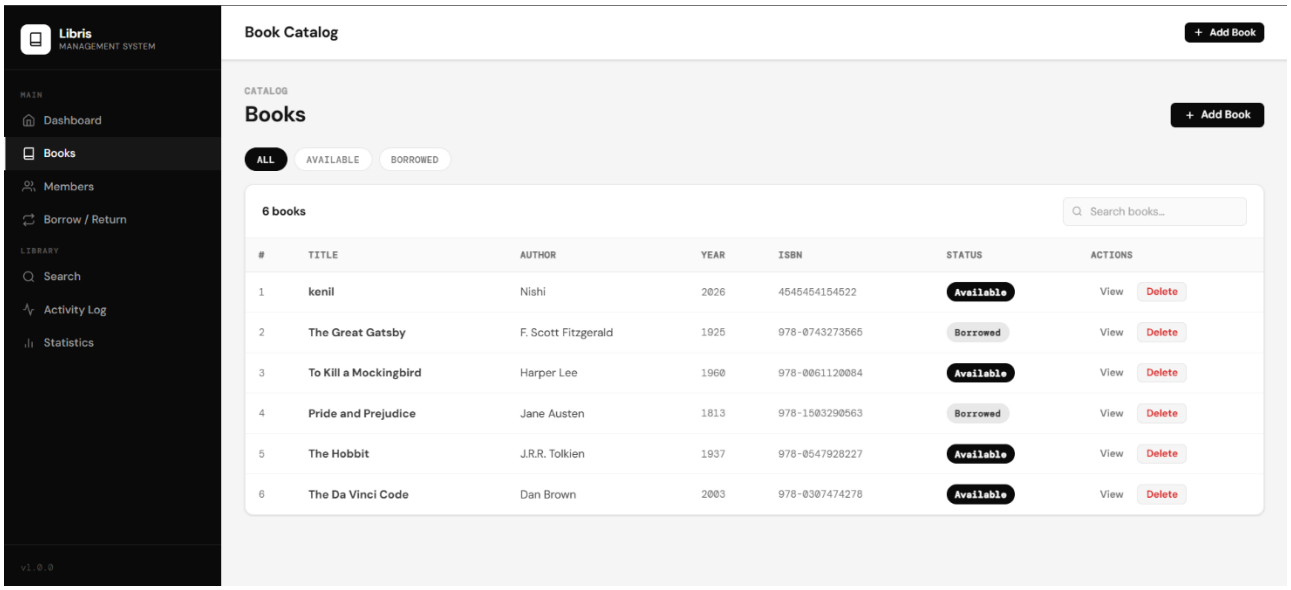

Figure 7.4 — Book Detail Page

7.5 Members Management Page

Route: `/users`

Lists all registered members with contact details and borrow counts.

Key Features:

- Member table: Name, ID, Email, Phone, Borrowed book count
- Delete button per member
- 'Add Member' navigation button

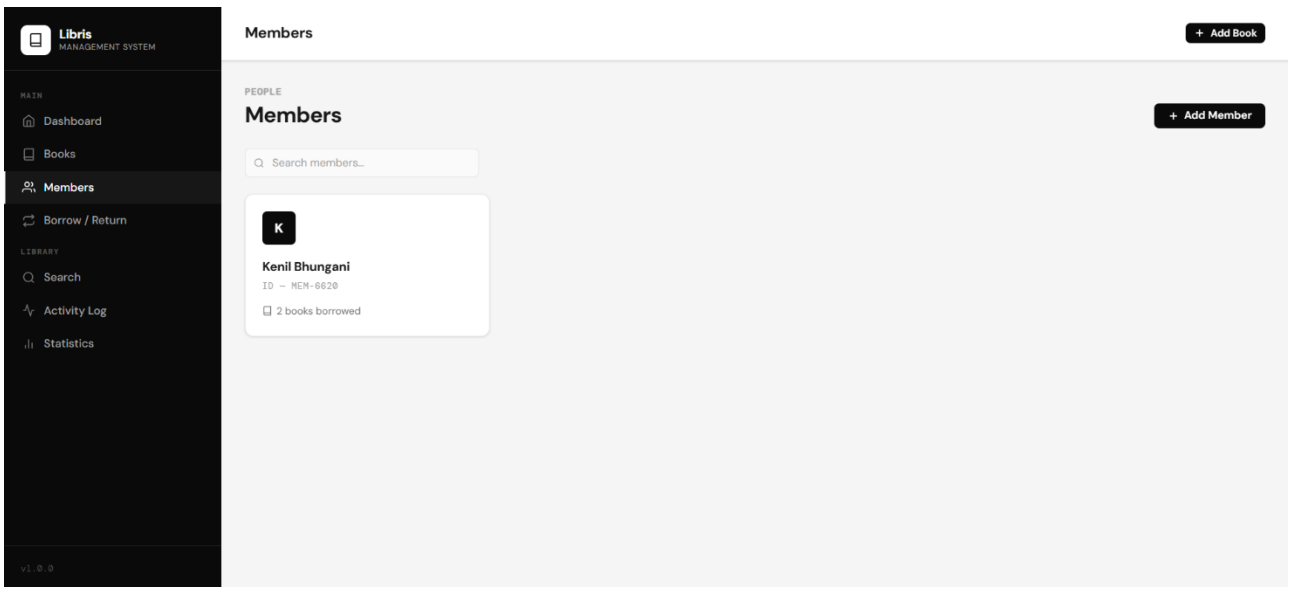

Figure 7.5 — Members Management Page

7.6 Add Member Page

Route: `/users/add`

Registration form for new library members.

Key Features:

- Required: Full Name, Member ID
- Optional: Email, Phone
- Client-side duplicate ID checking
- Success / error flash messages

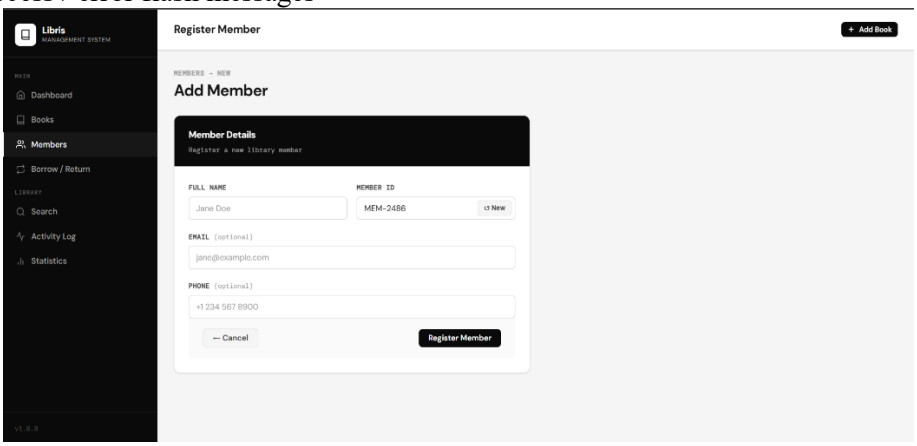

Figure 7.6 — Add Member Page

7.7 Borrow & Return Page

Route: `/borrow`

The operational core of Libris. Issues books and records returns.

Key Features:

- Available Books panel — all books open for borrowing
- Borrowed Books panel — all on-loan books with borrower names
- Issue Book form: select book, enter Member ID, click Issue
- Return Book form: select book, enter Member ID, click Return
- Flash messages confirming each action

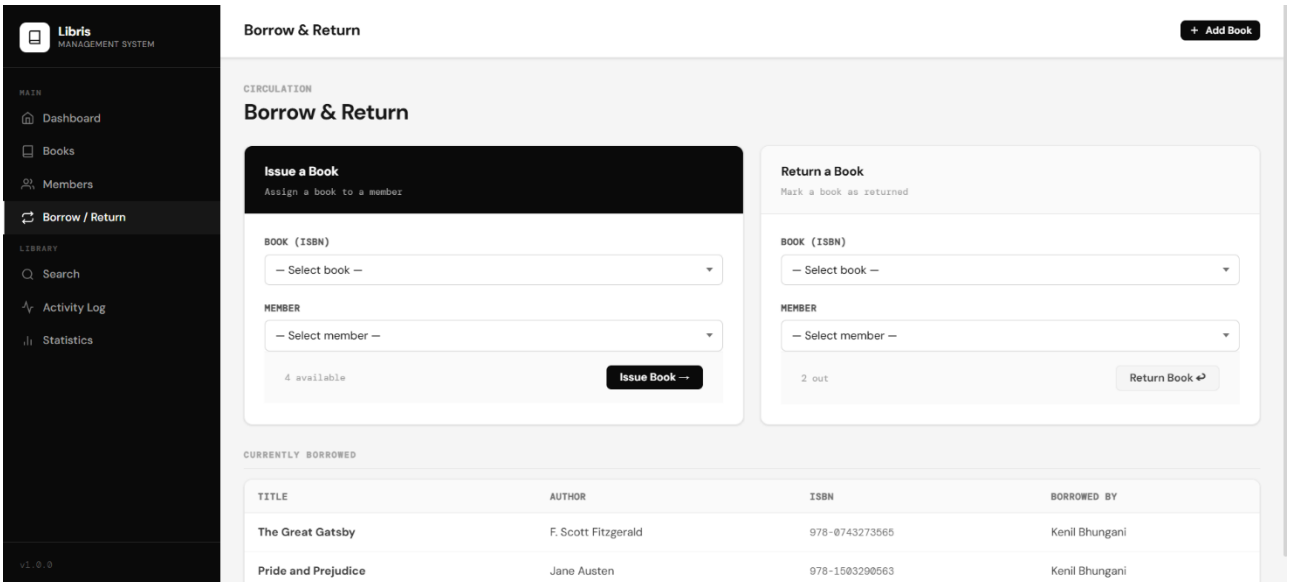

Figure 7.7 — Borrow & Return Page

7.8 Search Page

Route: `/search`

Search across the entire catalogue by any keyword.

Key Features:

- Search input field with submit button
- Results with full book details
- Partial and case-insensitive matching
- Result count displayed
- Links to individual book detail pages

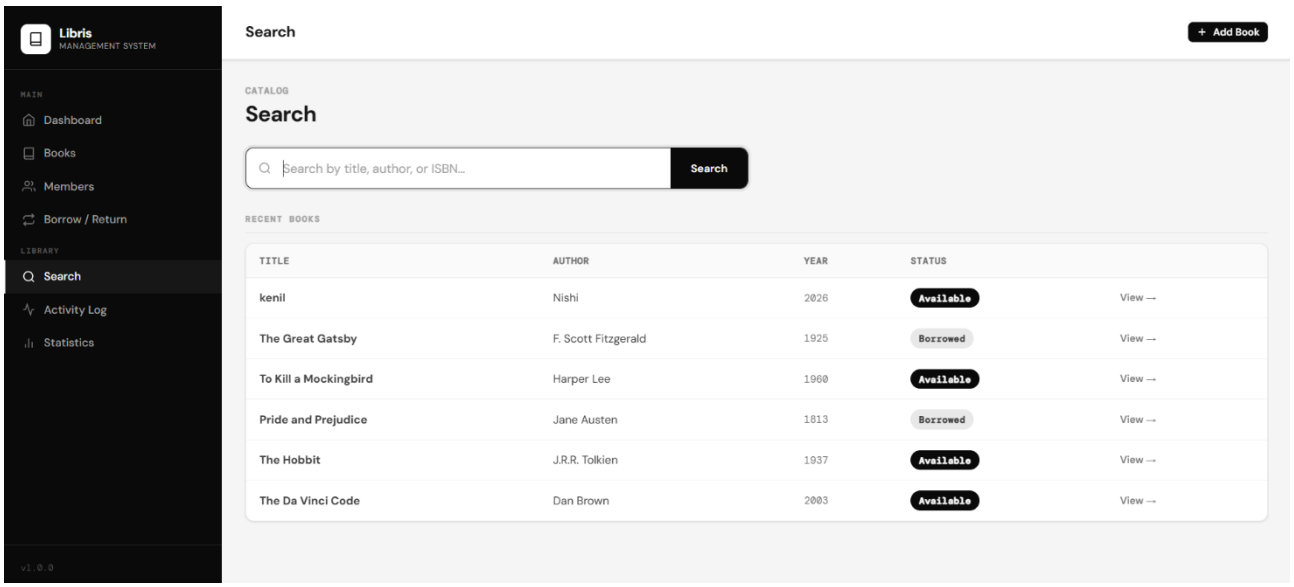

Figure 7.8 — Search Page

7.9 Activity Log Page

Route: `/activity`
Chronological record of all borrow, return, and addition events.

Key Features:

- Entries listed newest first
- Each entry: event type, member name, book title, date
- Log capped at 100 entries

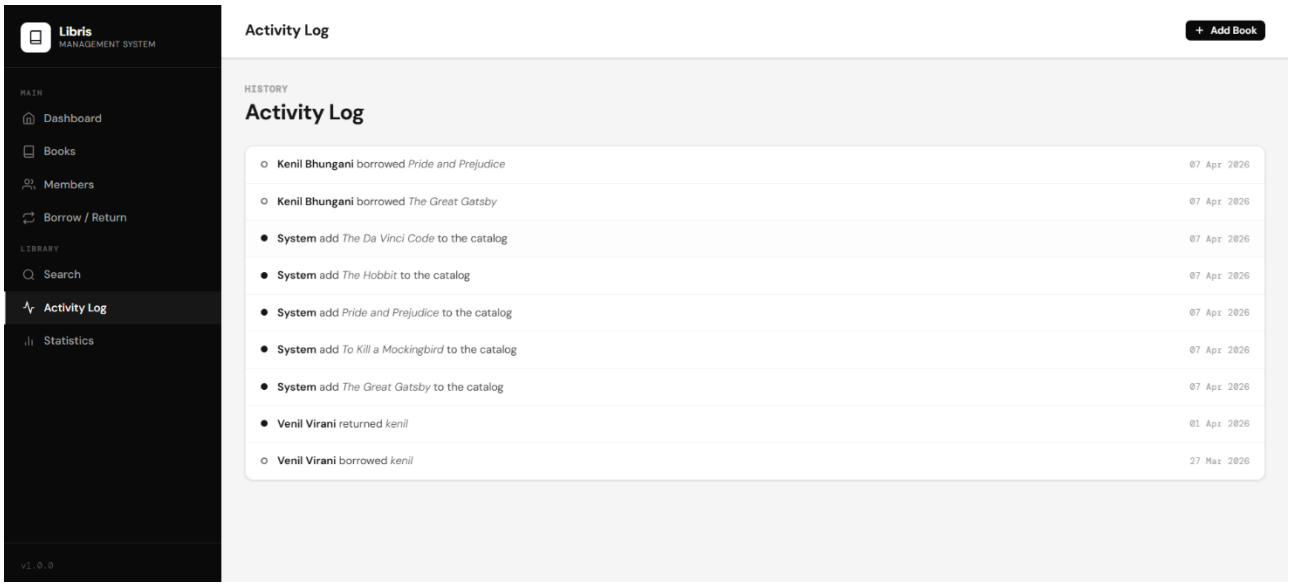

Figure 7.9 — Activity Log Page

7.10 Statistics Page

Route: `/stats`

Analytical insights into library usage patterns.

Key Features:

- Summary stats: Total, Available, Borrowed, Members
- Genre Breakdown: books per genre, sorted by most popular
- Top 5 Borrowers leaderboard from the activity log

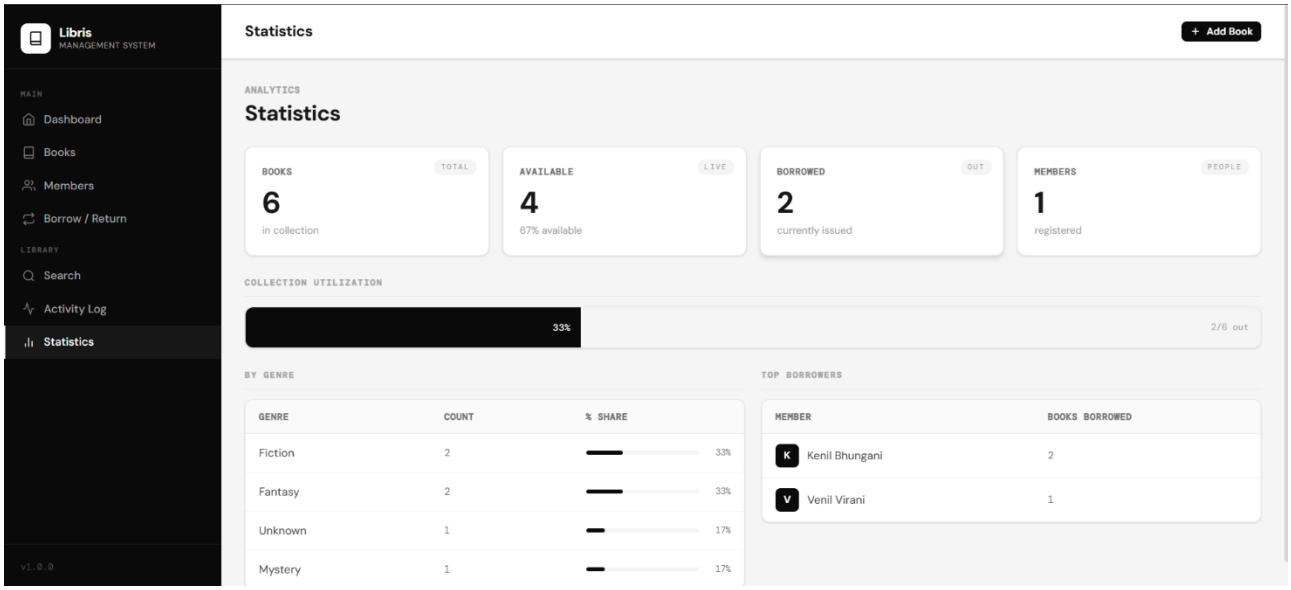

Figure 7.10 — Statistics Page

Chapter 8

Data Storage Reference

JSON schema documentation for all data files

8.1 books.json Schema

Field	Type	Required	Description
title	string	Yes	Full book title
author	string	Yes	Author's full name
year	string	Yes	Publication year (stored as string)
isbn	string	Yes	Unique ISBN identifier — primary key
genre	string	No	Genre or category label (may be empty)
description	string	No	Short description (may be empty)
available	boolean	Yes	true = available; false = currently borrowed
borrowed_by	string | null	Yes	Current borrower name, or null if available

```
[ { "title": "The Great Gatsby", "author": "F. Scott Fitzgerald", "year": "1925", "isbn": "9780743273565", "genre": "Fiction", "description": "A story of wealth and ambition.", "available": false, "borrowed_by": "Alice Johnson" } ]
```

8.2 users.json Schema

Field	Type	Required	Description
name	string	Yes	Full name of the member
user_id	string	Yes	Unique member ID — primary key
email	string	No	Email address (may be empty)
phone	string	No	Phone number (may be empty)
borrowed_books	array[string]	Yes	ISBNs of books currently borrowed

```
[ { "name": "Alice Johnson", "user_id": "U001", "email": "alice@example.com", "phone": "9876543210", "borrowed_books": ["9780743273565"] } ]
```

8.3 activity.json Schema

Field	Type	Values	Description
type	string	'borrow', 'return', 'add'	The type of library event
user	string	Member name or 'System'	Member involved, or 'System' for additions
book	string	Book title	Full title of the book involved
date	string	DD Mon YYYY	Event date, e.g. '10 Apr 2026'

```
[ { "type": "borrow", "user": "Alice Johnson", "book": "The Great Gatsby", "date": "10 Apr 2026" }, { "type": "add", "user": "System", "book": "The Great Gatsby", "date": "07 Apr 2026" } ]
```

Chapter 9

Frontend Reference

Stylesheet, JavaScript, and template hierarchy

9.1 style.css (26 KB)

The single global stylesheet linked in base.html, covering:

- **Layout & Grid:** Page wrappers, container widths, flexbox and grid layouts.
- **Navigation Bar:** Horizontal top nav with active link indicators.
- **Stat Cards:** Dashboard summary cards with hover effects.
- **Tables:** Alternating row colours, header styling, borders.
- **Forms:** Input fields, textareas, dropdowns, labels, and buttons.
- **Flash Messages:** Dismissible alert banners for success and error states.
- **Availability Badges:** Status pills — Available / Borrowed.
- **Responsive Design:** Media queries for tablet and mobile.

9.2 script.js (3 KB)

Adds client-side interactivity:

- **Flash Message Dismissal:** Allows users to manually close notification banners.
- **Form Validation:** Client-side checks before form submission.
- **Dynamic Dropdowns:** Filters dropdown options based on user selection.
- **Auto-dismiss Alerts:** Flash messages auto-dismiss after a timeout.

9.3 Jinja2 Template Hierarchy

All 11 templates extend base.html and override these blocks:

Block Name	Purpose
title	Sets the browser tab title for each page
content	Main page content — each child template fills this block
scripts	Optional extra JavaScript for page-specific needs

Example child template:

```
{% extends "base.html" %} {% block title %}Books — Libris{% endblock %} {% block content %} Book Catalogue ... page content ... {% endblock %}
```

Chapter 10

Code Quality — Trunk

Linting, formatting, and automated code checks

10.1 Overview

Libris uses Trunk (https://trunk.io) as its code quality platform. Trunk manages linters, formatters, and static analysis tools in a single unified configuration stored in the .trunk/ directory.

10.2 trunk.yaml

The main Trunk configuration file. Specifies enabled linters, formatters, versions, and custom settings. Last modified 08-04-2026.

10.3 Running Trunk

Command	Description
trunk check	Run all enabled linters and report issues
trunk fmt	Auto-format all source files
trunk check --all	Check all project files (not just changed files)
trunk upgrade	Upgrade Trunk and all managed tool versions

10.4 .trunk/.gitignore

Trunk's own .gitignore prevents tool binaries and generated output from being committed. Typically ignores the tools/, out/, and logs/ directories.

Chapter 11

Usage Guide

Step-by-step instructions for all core workflows

11.1 Adding a Book

- **Step 1:** Click **Books** in the navigation bar.
- **Step 2:** Click the **Add Book** button.
- **Step 3:** Fill in Title, Author, Year, and ISBN (required).
- **Step 4:** Optionally fill in Genre and Description.
- **Step 5:** Click **Add Book**. A success message confirms the addition.

Note: The ISBN must be unique. A duplicate ISBN will show a red error message.

11.2 Registering a Member

- **Step 1:** Click **Members** in the navigation bar.
- **Step 2:** Click the **Add Member** button.
- **Step 3:** Enter Full Name and a unique Member ID (e.g., U001).
- **Step 4:** Optionally enter Email and Phone.
- **Step 5:** Click **Register**. A success message confirms registration.

Note: The Member ID must be unique across all registered members.

11.3 Borrowing a Book

- **Step 1:** Click **Borrow / Return** in the navigation bar.
- **Step 2:** In the Issue Book form, select the book from the available list.
- **Step 3:** Enter the Member ID of the borrower.
- **Step 4:** Click **Issue Book**. The book status changes to 'Borrowed'.

Note: Only available books appear in the issue form. One book can only be borrowed by one member at a time.

11.4 Returning a Book

- **Step 1:** Click **Borrow / Return** in the navigation bar.
- **Step 2:** In the Return Book form, select the book from the borrowed list.
- **Step 3:** Enter the Member ID of the returning member.

- **Step 4:** Click **Return Book**. The book reverts to 'Available'.

11.5 Searching the Catalogue

- **Step 1:** Click **Search** in the navigation bar.
- **Step 2:** Type any keyword — title, author, ISBN, or genre.
- **Step 3:** Press Enter or click Search. Results appear below.
- **Step 4:** Click any result to view the full book detail page.

> *Note: Search is case-insensitive and supports partial matches.*

Chapter 12

Potential Improvements

Suggested enhancements for future versions

12.1 Short-Term

Due Date Tracking

Add a due_date field to borrow records and show overdue status on the dashboard.

Email Notifications

email_utils.py is present in the codebase — integrate it to send due date reminders to members.

Search Filters

Add filtering by availability, genre, or year on the search page.

Pagination

Add pagination to the books and members list pages for large catalogues.

Bulk Import

Add CSV import to bulk-add books from a spreadsheet.

12.2 Medium-Term

Database Backend

Replace JSON flat files with SQLite via SQLAlchemy for better performance and concurrent access.

User Authentication

Add login with admin and staff roles to protect administrative functions.

Book Cover Images

Allow cover image uploads stored in the static/ directory.

Barcode / ISBN Lookup

Integrate a barcode scanner or Open Library API to auto-fill book details.

Export Reports

Add PDF and CSV export for the catalogue, members list, and activity log.

12.3 Long-Term

REST API

Expose all functionality via a documented REST API for mobile apps and integrations.

Production Deployment

Deploy with Gunicorn + Nginx or containerise with Docker.

Fine Management

Track overdue books and calculate fines automatically.

Member Self-Service

Allow members to log in, view their loans, and search the catalogue themselves.

Chapter 13

Glossary

Definitions of key terms used in this document

Term	Definition
API	Application Programming Interface — a defined set of URL routes through which software components communicate.
Bytecode	Compiled Python source code (.pyc files) in __pycache__/ that speeds up subsequent imports.
Flask	A lightweight Python web framework providing URL routing, request handling, and Jinja2 integration.
GET	HTTP method used to retrieve data from the server without modifying it.
ISBN	International Standard Book Number — a unique numeric identifier for books, used as primary key in Libris.
Jinja2	A templating engine for Python used by Flask to generate HTML dynamically from template files.
JSON	JavaScript Object Notation — a lightweight text format for storing and exchanging structured data.
MVC	Model-View-Controller — a pattern separating data (Model), UI (View), and logic (Controller).
POST	HTTP method used to submit data to the server (e.g., form submissions that create or modify records).
Redirect	An HTTP response instructing the browser to navigate to a different URL — used after POST requests.
Route	A URL pattern mapped to a Python function in Flask via the @app.route() decorator.
Trunk	An all-in-one code quality platform managing linters, formatters, and static analysis tools.
WSGI	Web Server Gateway Interface — the Python standard for web server-application communication in production.

End of Document

Libris — Library Management System · Version 1.0.0 · April 2026 · Author: Bhungani Kenil Shaileshbhai